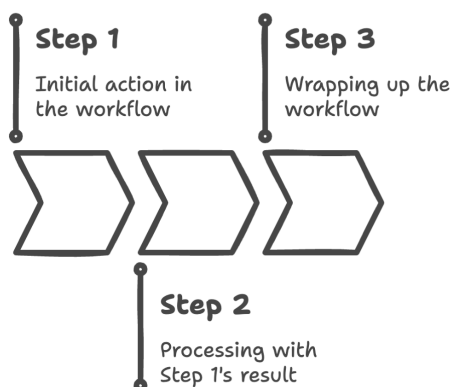


Designing multi-step workflows with custom events

Custom Events

Multiple steps are created by defining custom events that can be emitted by steps and trigger other steps. Let's define a simple 3-step workflow by defining two custom events, `FirstEvent` and `SecondEvent`. These classes can have any names and properties, but must inherit from `Event`.



! Note: Run the hidden cells below before running the rest of the code. !

```
!pip install llama-index -q -q
```

```
from llama_index.core.workflow import (
    Event,
    StartEvent,
    StopEvent,
    Workflow,
    step,
)

class FirstEvent(Event):
    first_output: str

class SecondEvent(Event):
    second_output: str
```



Defining the workflow

Now we define the workflow itself. We do this by defining the input and output types on each step.

- `step_one` takes a `StartEvent` and returns a `FirstEvent`
- `step_two` takes a `FirstEvent` and returns a `SecondEvent`
- `step_three` takes a `SecondEvent` and returns a `StopEvent`

```
class MyWorkflow(Workflow):
    @step
    async def step_one(self, ev: StartEvent) -> FirstEvent:
        print(ev.first_input)
        return FirstEvent(first_output="First step complete.")

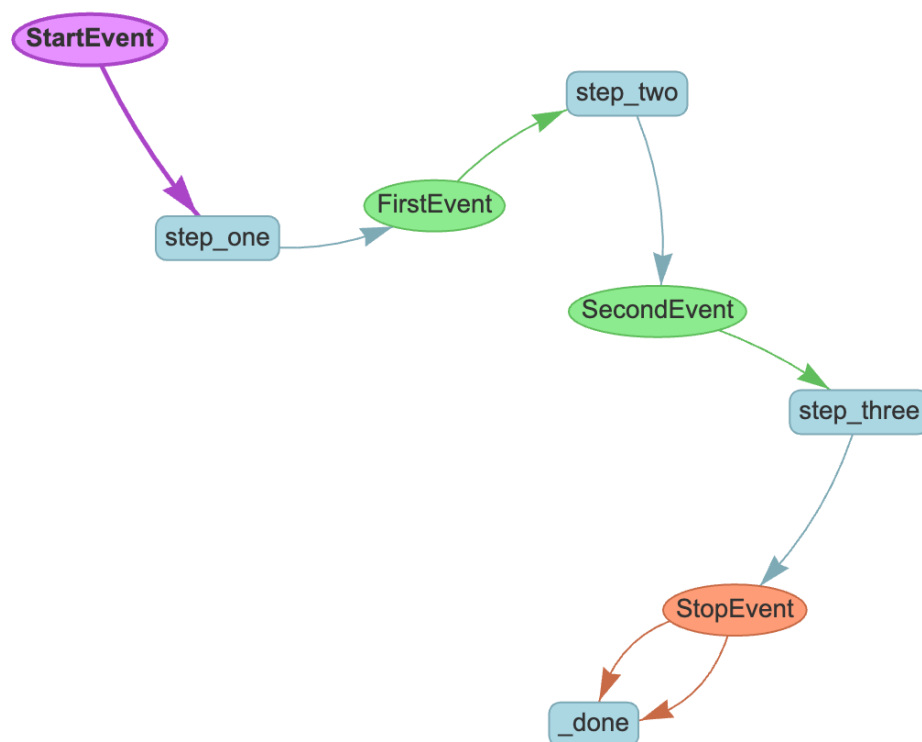
    @step
    async def step_two(self, ev: FirstEvent) -> SecondEvent:
        print(ev.first_output)
        return SecondEvent(second_output="Second step complete.")

    @step
    async def step_three(self, ev: SecondEvent) -> StopEvent:
        print(ev.second_output)
        return StopEvent(result="Workflow complete.")

w = MyWorkflow(timeout=10, verbose=False)
result = await w.run(first_input="Start the workflow.")
print(result)
```

```
Start the workflow.
First step complete.
Second step complete.
Workflow complete.
```

If we run this through the visualizer, we'll get something like this:



Looping

However, there's not much point to a workflow if it just runs straight through! A key feature of Workflows is their enablement of branching and looping logic, more simply and flexibly than graph-based approaches. To enable looping, we'll create a new `LoopEvent` (LoopEvent is not special, any event can be used to loop).

```
class LoopEvent(Event):
    first_input: str
```

Now we'll edit our `step_two` to make a random decision about whether to execute serially or loop back:

```
import random

class MyWorkflow(Workflow):

    # Step one will trigger on a StartEvent or a LoopEvent
    @step
    async def step_one(self, ev: StartEvent | LoopEvent) -> FirstEvent:
        print(ev.first_input)
        return FirstEvent(first_output="First step complete")

    # Step two returns either a SecondEvent or a LoopEvent
    @step
    async def step_two(self, ev: FirstEvent) -> SecondEvent | LoopEvent:
        print(ev.first_output)
        if random.randint(0, 1) == 0:
            print("Bad thing happened")
            return LoopEvent(first_input="Back to step one.")
        else:
            print("Good thing happened")
            return SecondEvent(second_output="Second step complete.")

    @step
    async def step_three(self, ev: SecondEvent) -> StopEvent:
        print(ev.second_output)
        return StopEvent(result="Workflow complete.")

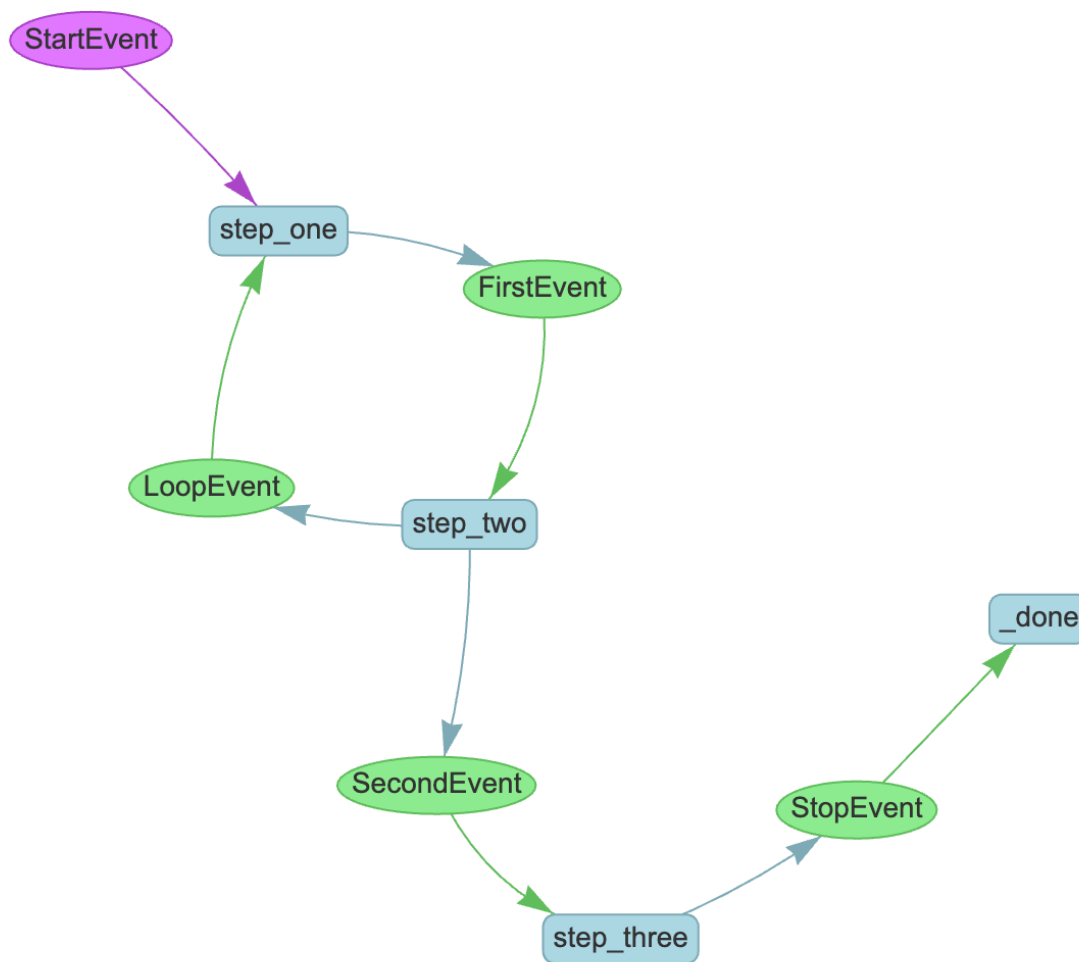
w = MyWorkflow(timeout=10, verbose=False)
result = await w.run(first_input="Start the workflow.")
print(result)
```

```
Start the workflow.
First step complete
Bad thing happened
Back to step one.
First step complete
Bad thing happened
Back to step one.
First step complete
Good thing happened
Second step complete.
Workflow complete.
```

Note the new type annotations on `step_one`: we now accept either a `StartEvent` or a `LoopEvent` to trigger the step. In step two, we emit either a `SecondEvent` or a `LoopEvent`.

In this run, the "bad" outcome happened once, repeating step one, then the "good" outcome happened, allowing the workflow to complete.

Our new, looping workflow visualizes like this:



Branching

The same constructs that allow us to loop allow us to create branches. Here's a workflow that executes two different branches depending on an early decision:

```
class BranchA1Event(Event):
    payload: str

class BranchA2Event(Event):
    payload: str

class BranchB1Event(Event):
    payload: str

class BranchB2Event(Event):
    payload: str

class BranchWorkflow(Workflow):
    @step
    async def start(self, ev: StartEvent) -> BranchA1Event | BranchB1Event:
        if random.randint(0, 1) == 0:
            print("Go to branch A")
            return BranchA1Event(payload="Branch A")
        else:
            print("Go to branch B")
            return BranchB1Event(payload="Branch B")

    @step
    async def step_a1(self, ev: BranchA1Event) -> BranchA2Event:
        print(ev.payload)
        return BranchA2Event(payload=ev.payload)

    @step
    async def step_b1(self, ev: BranchB1Event) -> BranchB2Event:
        print(ev.payload)
        return BranchB2Event(payload=ev.payload)

    @step
    async def step_a2(self, ev: BranchA2Event) -> StopEvent:
        print(ev.payload)
        return StopEvent(result="Branch A complete.")

    @step
    async def step_b2(self, ev: BranchB2Event) -> StopEvent:
        print(ev.payload)
        return StopEvent(result="Branch B complete.")
```

It visualizes like this:

